

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

BOA211

A winning combination: AWS Lambda and CQRS

Lukasz Panusz

(he/him)

Principal SA Team Lead (CEE)

Amazon Web Services

Wojciech Gawronski

(he/him)

Senior Developer Advocate (CEE)

Amazon Web Services



Session plan



1 hour session

10 minutes intro



Open discussion

Whiteboarding focus



Survey

Materials



Quick introduction...

**Command Query
Responsibility
Segregation (CQRS)**

Author: Greg Young
When: ~2010



**Command Query
Separation
(CQS)**

Author: Bertrand Meyer
When: ~2000



Let there be light bulb!



```
// TypeScript Code (Non CQS):
```

```
class LightBulb {  
  public readonly enabled: boolean;  
  
  constructor() {  
    this.enabled = false;  
  }  
  
  toggleLight() {  
    this.enabled = !this.enabled;  
    return this.enabled;  
  }  
}
```

```
// Usage:
```

```
const bulb = new LightBulb();  
const status = bulb.toggleLight();
```



[...] every method should either be a **command** that performs an action, or a **query** that returns data to the caller, but not both.

In other words, **asking a question should not change the answer.**

More formally, methods should return a value only if they are **referentially transparent** and hence possess **no side effects**.

- Bertrand Meyer

Author of the **Eiffel** (*programming language*)
and **Design by Contract** (*approach to software design*)

Command Query Separation (CQS)

```
// TypeScript Code (CQS):

class LightBulb {
  public readonly enabled: boolean;

  constructor() {
    this.enabled = false;
  }

  toggleLight() {
    this.enabled = !this.enabled;
  }

  isLightOn() {
    return this.enabled;
  }
}

// Usage:

const bulb = new LightBulb();
bulb.toggleLight();
const status = bulb.isLightOn();
```



Command Query Responsibility Segregation (CQRS)

Service
(query /command)

Data Model
(retrieval /mutation)



Evolution of CQS into
architecture
and data modelling
concepts



Beneficial for performance,
maintenance, complexity
management, resilience

Command Query Responsibility Segregation (CQRS)

```
// TypeScript Code (CQRS):
```

```
class ToggleLightCommand implements ICommand {
  protected readonly database: IDatabaseProvider;

  constructor(database) {
    this.database = database;
  }

  handle(bulbId): void {
    const state = this.database.get(bulbId);
    const updatedState = !state;
    this.database.update(bulbId, updatedState);
  }
}

class IsLightOnQuery implements IQuery<boolean> {
  protected readonly database: IDatabaseProvider;

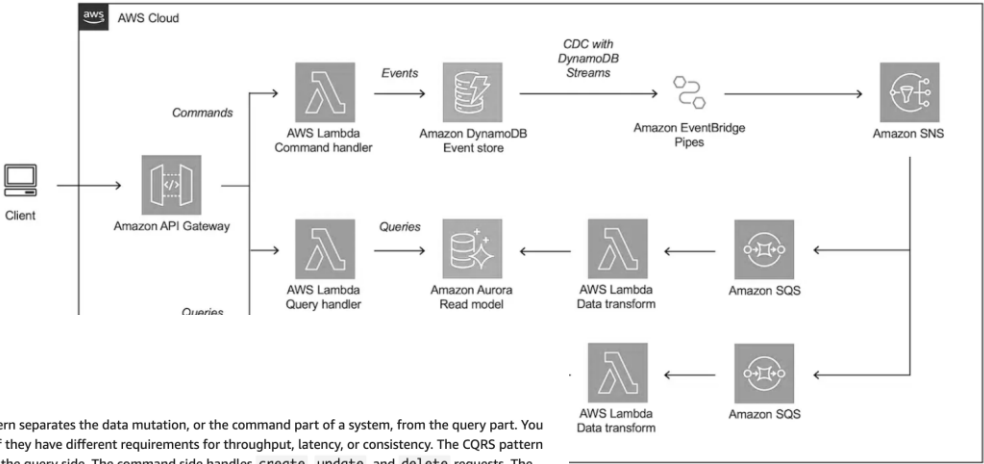
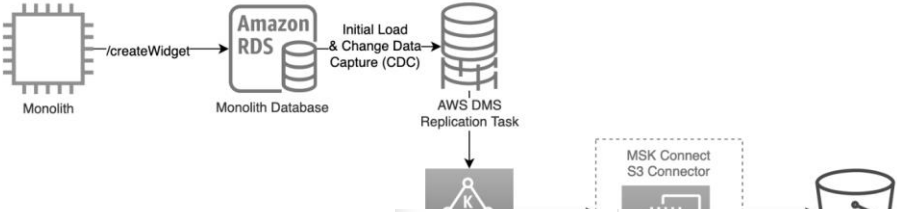
  constructor(database) {
    this.database = database;
  }

  handle(bulbId): boolean {
    const state = this.database.get(bulbId);
    return state;
  }
}
```



Build a CQRS event store with Amazon DynamoDB

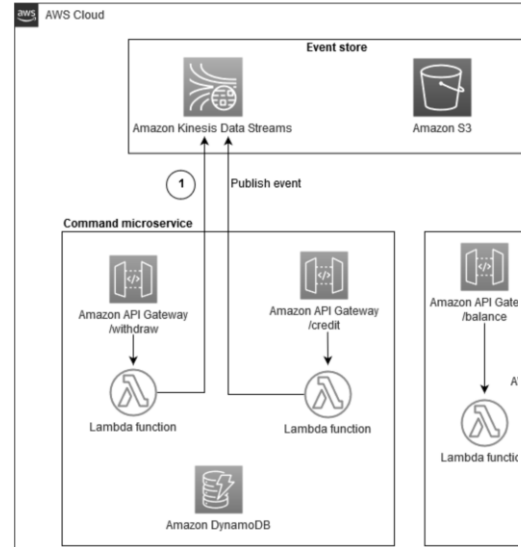
| on 26 SEP 2022 | in Amazon DynamoDB, Expert (400), Technical How-To | Permalink | Comments



Event sourcing pattern

PDF | RSS

In the following illustration, Kinesis Data Streams is the main component of captures application changes as events and persists them on Amazon Simple



AWS Prescriptive Guidance
Enabling data persistence in microservices

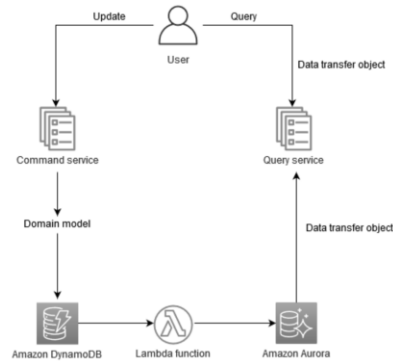
- Introduction
- Patterns for enabling data persistence
 - Database-per-service pattern
 - API composition pattern
 - CQRS pattern
 - Event sourcing pattern
 - Saga pattern
 - Shared-database-per-service pattern
- FAQ
- Resources
- Document history
- Glossary

CQRS pattern

PDF | RSS

The command query responsibility segregation (CQRS) pattern separates the data mutation, or the command part of a system, from the query part. You can use the CQRS pattern to separate updates and queries if they have different requirements for throughput, latency, or consistency. The CQRS pattern splits the application into two parts: the command side and the query side. The command side handles create, update, and delete requests. The query side runs the query part by using the read replicas.

In the following illustration, a NoSQL data store, such as DynamoDB, is used to optimize the write throughput and provide flexible query capabilities. This achieves high write scalability on workloads that have well-defined access patterns when you add data. A relational database, such as Aurora, provides complex query functionality. A DynamoDB stream sends data to a Lambda function that updates the Aurora table.



AWS Database Blog

Modernize legacy databases using event sourcing and CQRS with AWS DMS

| on 15 SEP 2022 | in Amazon Managed Streaming For Apache Kafka (Amazon MSK), AWS Database Migration Service, Expert (400) | Permalink | Comments | Share

h Event Sourcing,

fits of using AWS serverless services to

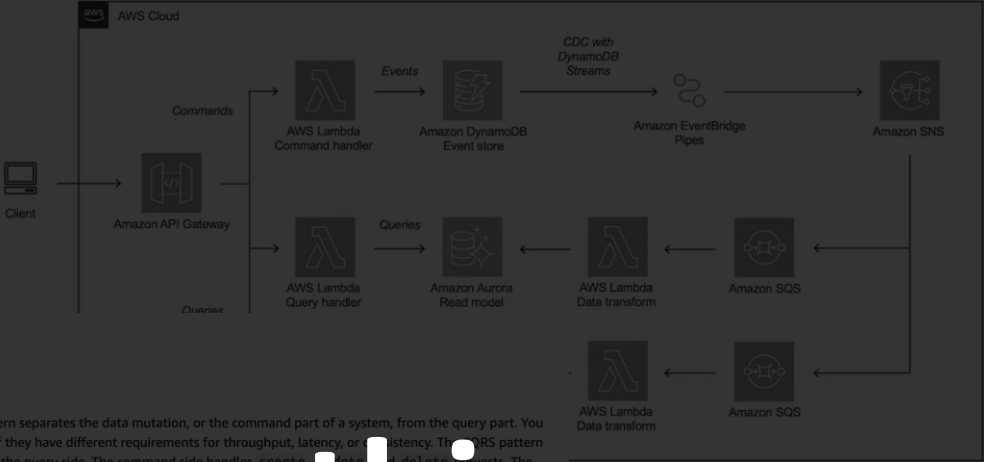
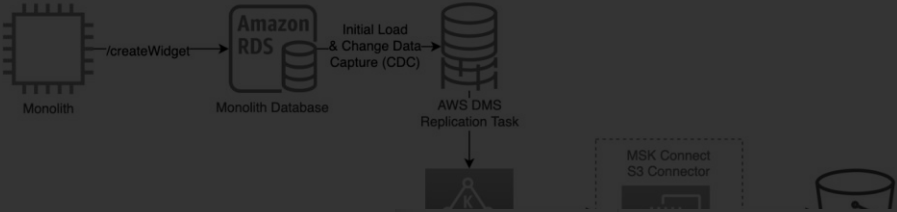
event-driven	aws
ommand ynamoDB	DynamoDB Streams
	Lambda function
	3

oservice (serverless)



Build a CQRS event store with Amazon DynamoDB

| on 26 SEP 2022 | in Amazon DynamoDB, Expert (400), Technical How-To | Permalink | Comments



AWS Prescriptive Guidance

CQRS pattern

Let us clarify one thing:
it is not about infrastructure

Event sourcing pattern

PDF | RSS

In the following illustration, Kinesis Data Streams is the main component of captures application changes as events and persists them on Amazon Simple



Event sourcing pattern

Command service

Query service

Domain model

Data transfer object

Modernize legacy databases using event sourcing and CQRS with AWS DMS

| on 15 SEP 2022 | in Amazon Managed Streaming For Apache Kafka (Amazon MSK), AWS Database Migration Service, Expert (400) | Permalink | Comments | Share

serverless (serverless)



Let's talk about the process...



As a **1st step** we will do refactoring to the CQRS on the **code** and **architecture** levels



Then in a **2nd step** we will build on that and design **infrastructure** level via *serverless* service offering

It's time to start!



github.com/build-on-aws/aws-lambda-and-cqrs-a-winning-combination

Summary!



Materials



github.com/build-on-aws/aws-lambda-and-cqrs-a-winning-combination

Stay connected with AWS!



Community.AWS



AWS Skill Builder



@awsdevelopers



@aws-developers



@awsdevelopers



collectives/aws

Thank you!



Please complete the session survey in the mobile app



Lukasz Panusz



pnszlp@amazon.com



/in/lukasz-panusz



Wojciech Gawronski



wojgaw@amazon.com



/in/afronski